

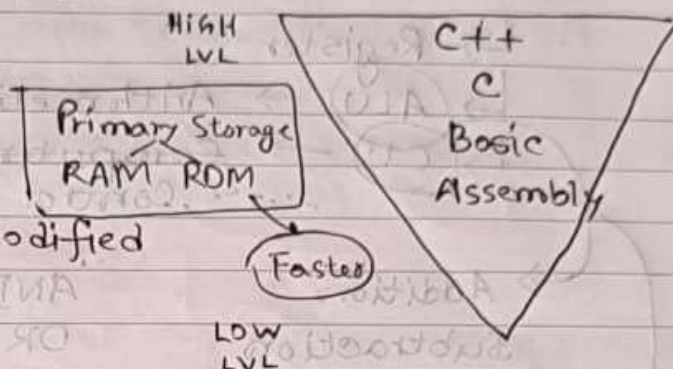
Arc Types

- ① Bonne Newmann Arc
- ② Harvard
- ③ Modified Harvard
- ④ RISC/CISC

RISC Processor → Modified

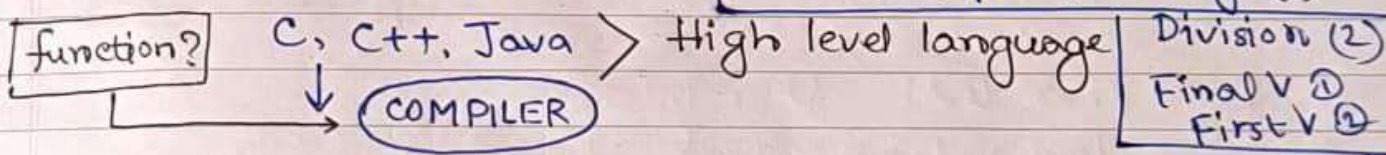
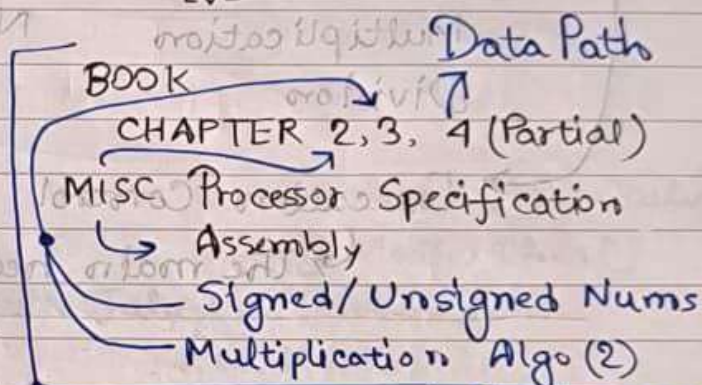
CPU, GPU

Processor ← General term



- ① Register (32bits)
- ② Assembly

- ① High level language
- ② Symbolic language
- ③ Machine language



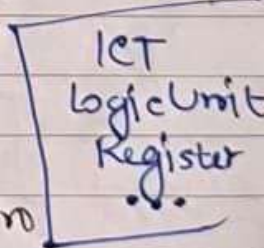
Symbolic language > Assembly language

Machine language/binary > Low level language

- ① Von Neuman
- ② Harvard
- ③ Modified Harvard
- ④ RISC & CISC

RISC
Reduced Instruction
Set Computing

CISC
Complex Instruction
Set Computing



ASSEMBLY TO 'C'
'C' to ASSEMBLY
CONVERSION

CT
Mid

Volatile → To be destroyed

cpu

↳ Register

↳ ALU

↳ CU

→ Arithmetic Logic Unit

→ ~~Computing~~ Unit
Control

→ Addition

Subtraction

Multiplication

Division

AND

OR

NOT

→ Execution Control

↳ The main heart of CPU

↓
COMPIER

↓
ASSEMBLER

MIPS
Microprocessor
without Interlocked
Pipelined
Stages

RISC & CISC A923

High Level
↓

Assembly
↓

Processor Architecture

Machine language

MIPS

① Register file

Instruction
would be 32bits

② Memory

Registers are also
32

③ Instruction

General Purpose

Register

In assembly,

register is indicated with "\$"

\$0

\$31

Size of each register

32bits / 4 bytes / 1 word

Special purpose registers — PC

HI

LO

size: also 32bit

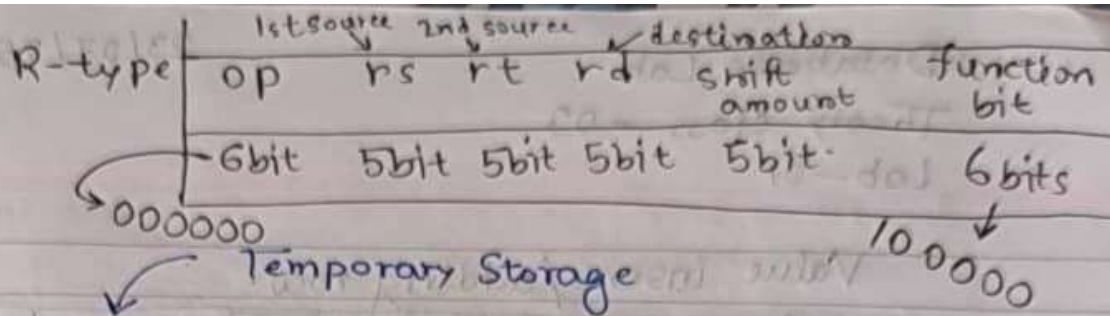
PC: Program Counter

↳ next instruction's

memory address hold করে

HI: Stores higher bits (উচ্চ অংশ)

LO: Stores least significant bits (নিম্ন অংশ)



Num
\$0 START

\$8 \$15

\$16 \$23

\$31 END

to - t7

s0 - s7

↑
save register

$$f = a + b - c + d$$

$$a = \$\Delta 1$$

$$b = \$\Delta 2$$

$$c = \$\Delta 3$$

$$d = \$\Delta 4$$

OP code
Operational code

clock cycle

Memory

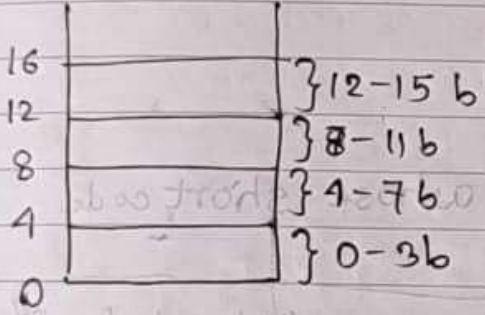
MIPS processor is word addressible

4 byte = 1 word

$$\$t_0 = \$s_1 + \$s_2$$

$$\$t_1 = \$s_3 + \$s_4$$

$$t = \$s_5 = \$t_0 - \$t_1$$



Memory to Register

LOAD

in assembly [LW]

Register to Memory

STORE

in assembly []

Instruction

This is also 32 bits

Instruction format

① R-type register ← register

② I-type immediate

③ J-type jump

load, store, addi, beq, bne

Arithmetic & Logical

Memory related

Goto a special instruction [Add]

memory addresses, Immediate value or registers

Only registers

① Opcode → instruction code
② Operand

$$\$s1 = \$s2 + \$s3$$

$$\text{Add } \$s1, \$s2, \$s3$$

operands

27-07-25

Computer Architecture
class - 03 | Chapter II

Def Machine lang

Def Assembly lang

= MIPS is a general purpose register, load-store,
fixed instruction-length architecture.

Smaller is faster

32bit $\gg$ 64bits

MIPS is optimized for fast pipelined performance,
not for low instruction count.

MIPS performs faster

A special purpose 32bit program counter.

4 byte = 32bit = 1 word

\$8-\$15 $\rightarrow$ temp data, not preserved by subprograms

\$16-\$23 $\rightarrow$ Saved registers, " " " "

\$t0 $\rightarrow$ \$8

\$s0 $\rightarrow$ \$16

MIPS $\rightarrow$ \$0 \$zero $\rightarrow$ hardwired 0 "0bit"

$\hookrightarrow$ cannot be overwritten

$\hookrightarrow$ useful for common operations
"move"

`add $t2 $t0, $t1`

Memory Organization

$\hookrightarrow$ Word conversion

Q1 $\rightarrow$ why word addressible?

Memory Operands

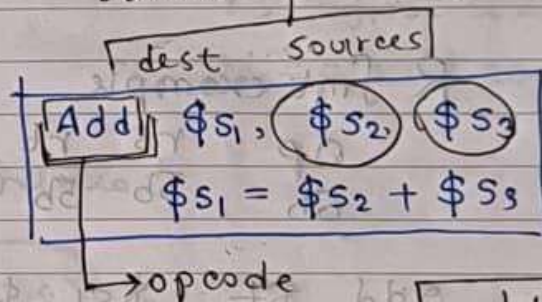
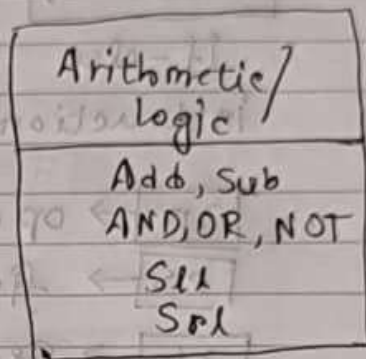
why load-store architecture
 store ~~load~~ → reg to mem
 load → mem to reg

Registers vs Memory

internal
faster

external
slower

compiler must
use reg for
variables as much
as possible



Opcode → Operational Code
 Operand

source → register, immediate value, mem address
 destination → register, mem address

high
 ↓
 assembly
 ↓
 machine

Instruction format

Name	Bit Fields	Notes
R fmt	6bits op, 5bits rs, 5bits rt, 5bits rd, 5bits shmt, 6bits shmt	Arithmetic, logic
I fmt	op, rs, <u>rt</u> , address/immediate (16)	load/store, branch, immediate
J R fmt	op, target, address (26)	Jump

R → register

I → immediate

J → jump

i
fmt

beq → branch if equal

bne → branch if not equal

Addi

lw
sw

bne, beq

if-else

instruction

Op

→ op code (000000 for R fmt)

rs

→ first source register number

rt

→ second source register number

rd

→ destination register number

R fmt example

Op	rs	rt	rd	shamt	func
6b	5b	5b	5b	5b	6b

Add \$t0, \$s1, \$s2

Special

\$s1

\$s2

\$t0

0

add

0

17

18

8

0

32

000000

10001

10010

01000

00000

100000

I fmt example

Op

rs

rt

rd

shamt

func

Op

rs

rt

rd

shamt

func

MIPS addressing modes

Register addressing

↳ operands are in a register
eg. `add $3, $4, $5`

Immediate Addressing

↳ `Addi $s1, $s2, 16`

rd = destination

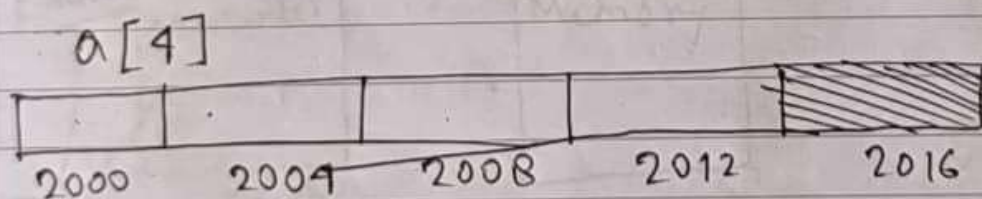
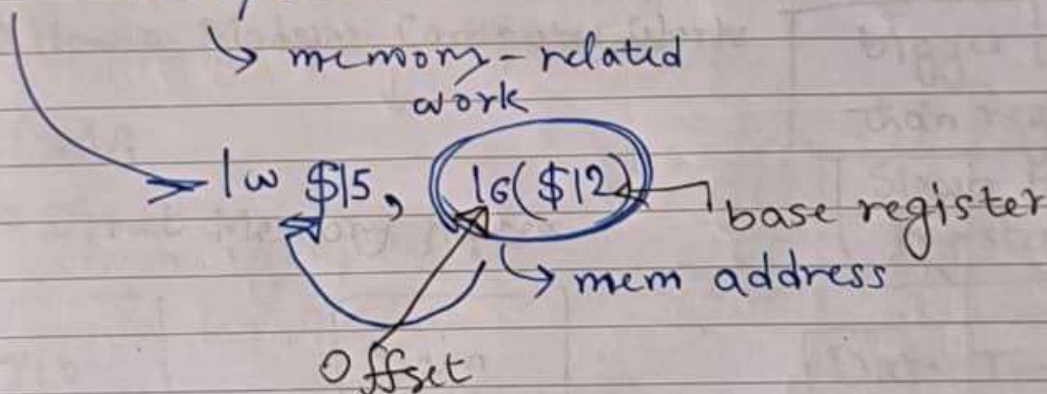
rs = first source

rt = second source

Base Addressing

↳ load/store

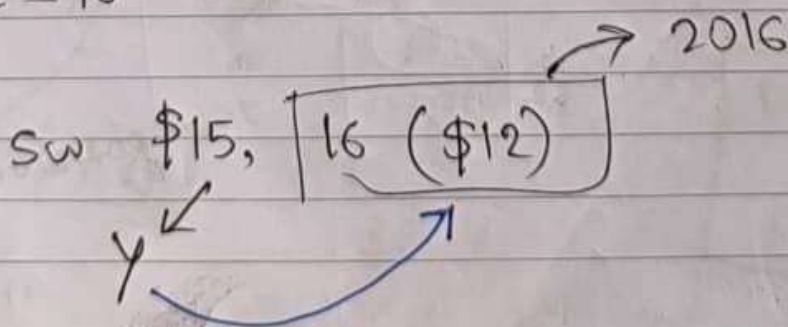
↳ memory-related work



$ba = 2000$
 $offset = 16$ $\rightarrow$ mem address = 2016



Immediate
fmt



beq \$0, \$5, label

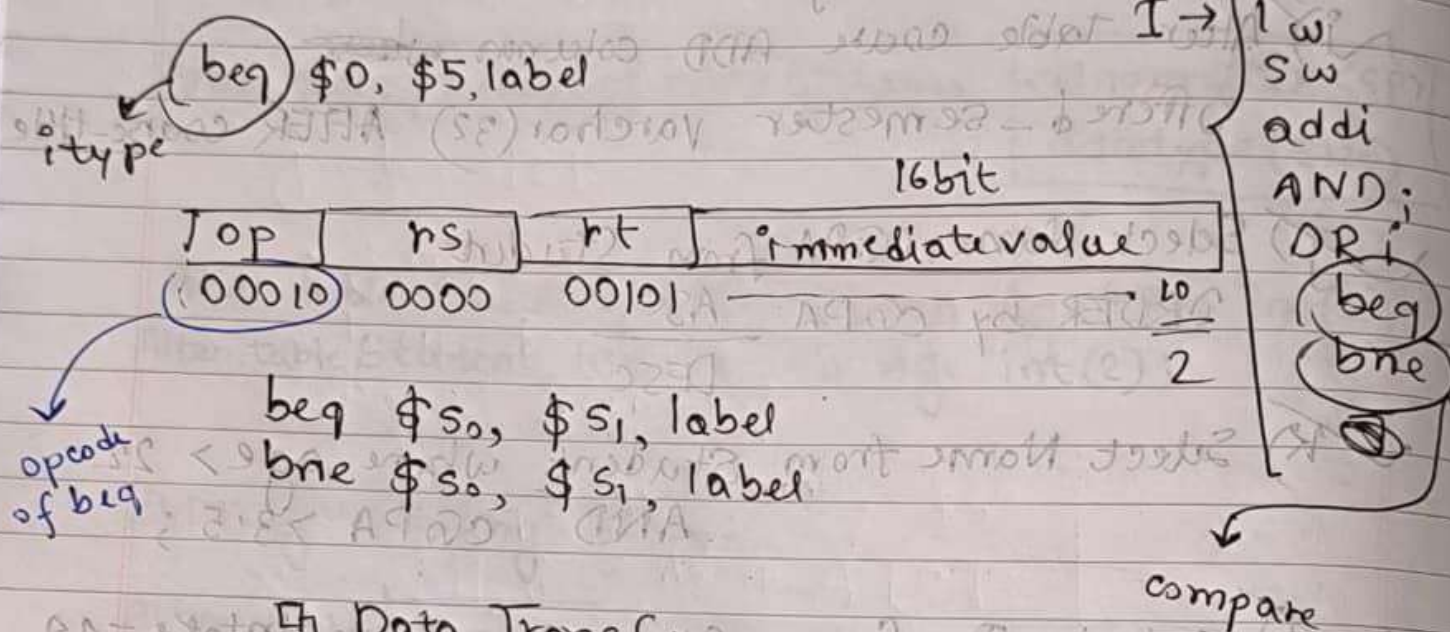
i format

if $\$5 == \0

→ $\$5 == 0$

mem address

Detail of MIPS PC-Relative



Data Transfer

Instruction	Example	Meaning	Comment
load word	lw \$1, 100(\$2)	\$1 = Mem(\$2 + 100)	
store word	sw \$1, 100(\$2)	Mem(\$2 + 100) = \$1	
load upper immediate	lwi \$1, label		
load immediate	li \$1, 100		

Arithmetic Example

C code:

$f = (g + h) - (i + j);$

Where $f-j$ is in $\$s0$ $\$s4$

Compiled MIPS code:

```
add $t0, $s1, $s2;
add $t1, $s3, $s4;
sub $s0, $t0, $t1;
```

MIPS
code
conversion

C code:

$a = b + c$

$\$s0$
 $\$s1$ $\$s2$
add $\$s3, \$s1, \$s2$

$f = \$s0$

$g = \$s1$

$h = \$s2$

$i = \$s3$

$j = \$s4$

R-type

Arithmetic

and
logic

add, sub,

AND,

OR,

NOT,

Sll, $\rightarrow$ shift left logical

Srl $\rightarrow$ shift right logical

Memory Operands

Load instruction

- lw refers to load word + $r = [\$r]$
- lw registerName, (offset) (reg With BA)
- lw $\$t0, 8(\$s3)$

base address

offset

Memory Operand Example I (array)

C code:

$g = h + A[8];$

• g in $\$s1$

h in $\$s2$

ba of A in $\$s3$

Next
Sunday
Exam

$g = h + A[8];$

$lw \$t0, 32(\$s3)$
 $add \$s1, \$s2, \$t0$

mem-address = t
 $\hookrightarrow 4 * \text{offset} + \text{base address}$
 $= 4 * 8 + \$s3$
 $T = 32 + \$s3$

C code:

$A[12] = h + A[8]$

h in $\$s2$ BA(A) in $\$s3$

$lw \$t0, 32(\$s3)$

~~$lw \$t1, 48(\$s3)$~~

$add \$s1, \$s2, \$t0$

~~$lw \$t1, 48(\$s3)$~~

$sw 48(\$s3), \$s1$

$\$t0 \rightarrow 96, 98(\$s3)$

mem-address

$4 * 8 = 32 + \$s3 \quad 32(\$s3)$

$4 * 12 = 48 + \$s3 \quad 48(\$s3)$

logical operations

Operation C Java MIPS
shift left logical <<

sll

elaboration

r type

opcode	rs	rt	rd	shift amount	fn
X	\$s0	\$t2		3	

binary

Shift operations
प्रश्न exam